IS LAB D	Gudowicz Michał Album 55684 Grupa 437	**Wersja 2026-09-14**

WPROWADZENIE DO PROGRAMOWANIA W PHP

SPIS TREŚCI

Spis treści....1
Cel zajęć....1
Uwaga....1
Pobranie i uruchomienie PHP....2
Instalacja Node.js i LESS....3
Pobranie i konfiguracja frameworka....4
XDEBUG....6
Zatrzymanie w pułapce....8
Omówienie frameworka....9
Rozszerzenie aplikacji....9
Commit projektu do GIT....12
Podsumowanie....12

CEL ZAJĘĆ

Celem głównym zajęć jest zdobycie następujących umiejętności:

- konfiguracji środowiska do programowania w PHP;
- połączenia z bazą danych;
- konfiguracji narzędzi do debugowania;
- analizy istniejących systemów webowych bez dokumentacji;
- uzupełniania istniejących systemów webowych o nowe funkcjonalności.

W praktycznym wymiarze uczestnicy zapoznają się z istniejącym naiwnym frameworkiem do tworzenia systemów webowych i uzupełnią go o nową funkcjonalność.

UWAGA

Ten dokument aktywnie wykorzystuje niestandardowe właściwości. Podobnie jak w LAB A wejdź do `Plik` -> `Informacje` -> `Właściwości` -> `Właściwości zaawansowane` -> `Niestandardowe` i zaktualizuj pola. Następnie uruchom ten dokument ponownie lub `Ctrl+A` -> `F9`.

POBRANIE I URURCHOMIENIE PHP

Zaloguj się do systemu Windows / pulpitu zdalnego rdp.wi.zut.edu.pl:

- spoza sieci ZUT potrzebny VPN: https://uci.zut.edu.pl/uslugi-uci/vpn.html;
- nazwa użytkownika: `WIAD\ab12345`
- komputer: `rdp.wi.zut.edu.pl`

Odwiedź stronę https://windows.php.net/download/. Pobierz `PHP 8.5.* x64 NTS`. Uwaga! Użyj linku „Zip” a nie „~~Development package~~”.

Wypakuj pobrane archiwum do `I:\php-8.5.10-nts-Win32-vs17-x64` (ścieżka może się nieznacznie różnić – jeśli tak, edytuj właściwości tego dokumentu, żeby wszystkie ścieżki poniżej nadawały się do kopiuj-wklej).

Otwórz panel sterowania. W polu wyszukiwania wpisz `path`. Wybierz edycję zmiennych środowiskowych użytkownika. Znajdź zmienną `Path` i kliknij edycję. Dodaj ścieżkę `I:\php-8.5.10-nts-Win32-vs17-x64`.

UWAGA! W toku studiów być może do Path zostały dodane inne wersje PHP lub XAMPP. Za kilka kroków sprawdzana będzie wersja PHP – upewnij się, że faktycznie wiesz z której instancji PHP korzystasz przez resztę tego laboratorium 😉.

Skopiuj plik `I:\php-8.5.10-nts-Win32-vs17-x64\php.ini-development` jako `php.ini`, po czym edytuj jego zawartość – odkomentuj poniższe ustawienia:

```
extension_dir = "ext"
...
extension=curl
extension=gd
extension=intl
extension=mbstring
extension=openssl
extension=pdo_sqlite
```

Otwórz terminal PowerShell i wejdź do katalogu laboratoriów. **Uwaga! Nie działa na innych terminalach.**

Wykonaj komendę

```
php -i | Select-String -Pattern '(PHP Version)|(extension_dir)|(OpenSSL
support)|(PDO drivers)|(GD Support)|intl|(cURL support)|multibyte'
```

Oczekiwany wynik:

```
PHP Version => 8.5.10
Zend Multibyte Support => provided by mbstring
PHP Version => 8.5.10
extension_dir => ext => ext
zend.multibyte => Off => Off
cURL support => enabled
GD Support => enabled
intl
intl.default_locale => no value => no value
intl.error_level => 0 => 0
intl.use_exceptions => Off => Off
Multibyte Support => enabled
Multibyte string engine => libmbfl
Multibyte (japanese) regex support => enabled
Multibyte regex (oniguruma) version => 6.9.10
OpenSSL support => enabled
PDO drivers => sqlite
OpenSSL support => enabled
```

Zastąp poniższy prostokąt swoim zrzutem ekranu:

```
Zend Multibyte Support ⇒ provided by mbstring
PHP Version ⇒ 8.5.12
extension_dir ⇒ /opt/homebrew/lib/php/pecl/20250925 ⇒ /opt/homebrew/lib/php/pecl/20250925
zend.multibyte ⇒ Off ⇒ Off
cURL support ⇒ enabled
Multibyte decoding support using mbstring ⇒ enabled
GD Support ⇒ enabled
intl
intl.default_locale ⇒ no value ⇒ no value
intl.error_level ⇒ 0 ⇒ 0
intl.use_exceptions ⇒ Off ⇒ Off
Multibyte Support ⇒ enabled
Multibyte string engine ⇒ libmbfl
Multibyte (japanese) regex support ⇒ enabled
Multibyte regex (oniguruma) version ⇒ 6.9.10
OpenSSL support ⇒ enabled
PDO drivers ⇒ dblib, mysql, odbc, pgsql, sqlite
Multibyte character support ⇒ enabled
Native OpenSSL support ⇒ enabled
sqlite3.extension_dir ⇒ no value ⇒ no value
```

Punkty:	0	1

Instalacja Node.js i LESS

Ta sekcja częściowo mogła zostać zrealizowana podczas wcześniejszych laboratoriów. Upewnij się, że zainstalowana jest w miarę bieżąca wersja Node.js.

Wejdź na stronę https://nodejs.org/en/download/current. Pobierz wersję Current -> Windows x64 -> Standalone Binary (.zip). Rozpakuj archiwum do katalogu pobranych, przykładowo C:\Users\akarczmarczyk\Downloads\node-v24.21.0-win-x64. Dodaj ten folder do zmiennej środowiskowej Path użytkownika.

Uwaga! Rozpakowywanie node do pobranych to zła praktyka. Natomiast w związku z problemem z wydajnością node'a na zasobie zdalnym I:, musimy upchnąć binarki gdzieś na dysku C. Ważne – katalogi na dysku C nie mają gwarantowanych kopii zapasowych.

Uruchom nowe okno wiersza poleceń. Wykonaj polecenia `node -v` i `npm -v`. Powinno zadziałać – udało się zainstalować NODE i NPM lokalnie dla użytkownika.

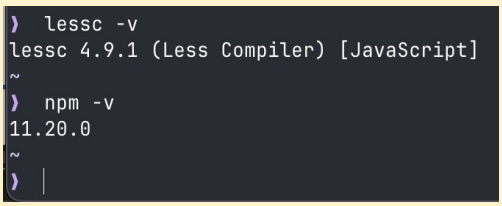
Następnie zainstalujemy Less. Wykonaj polecenie:

```
npm install -g less less-plugin-clean-css
```

Sprawdź poprawność instalacji:

```
lessc -v
```

Wstaw zrzut ekranu wyniku działania poleceń `npm -v` i `lessc -v`



Punkty:	0	1

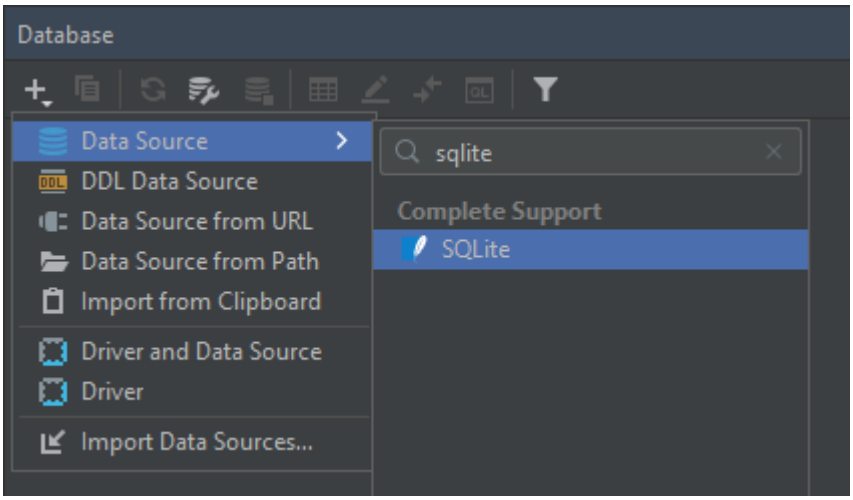
POBRANIE I KONFIGURACJA FRAMEWORKA

Sklonuj naiwny framework z Githuba: https://github.com/ideaspot-pl/custom-php-framework .

Wejdź do pobranego katalogu i skompiluj style:

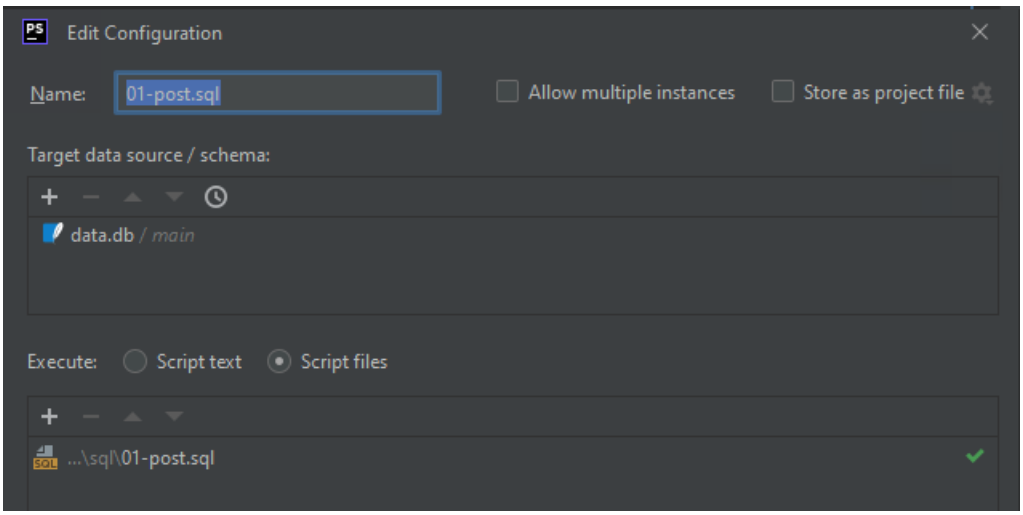
```
lessc public\assets\src\less\style.less public\assets\dist\style.min.css --clean-css --source-map
```

Otwórz projekt w PhpStorm. W głównym katalogu projektu stwórz pusty plik `data.db`. Rozwiń panel `Database` z prawej strony. Dodaj nowe źródło danych SQLite:



W sekcji File kliknij przycisk `...` i odszukaj plik `data.db` w katalogu głównym frameworka. Pobierz brakujące sterowniki, jeśli PhpStorm o to poprosi. Zatwierdź dodawanie źródła danych przyciskiem `OK`.

Kliknij prawym przyciskiem w drzewie projektu na `sql/01-post.sql` i wybierz z menu pozycję `Run (…)`. Dostosuj konfigurację jak poniżej i kliknij `Run`.

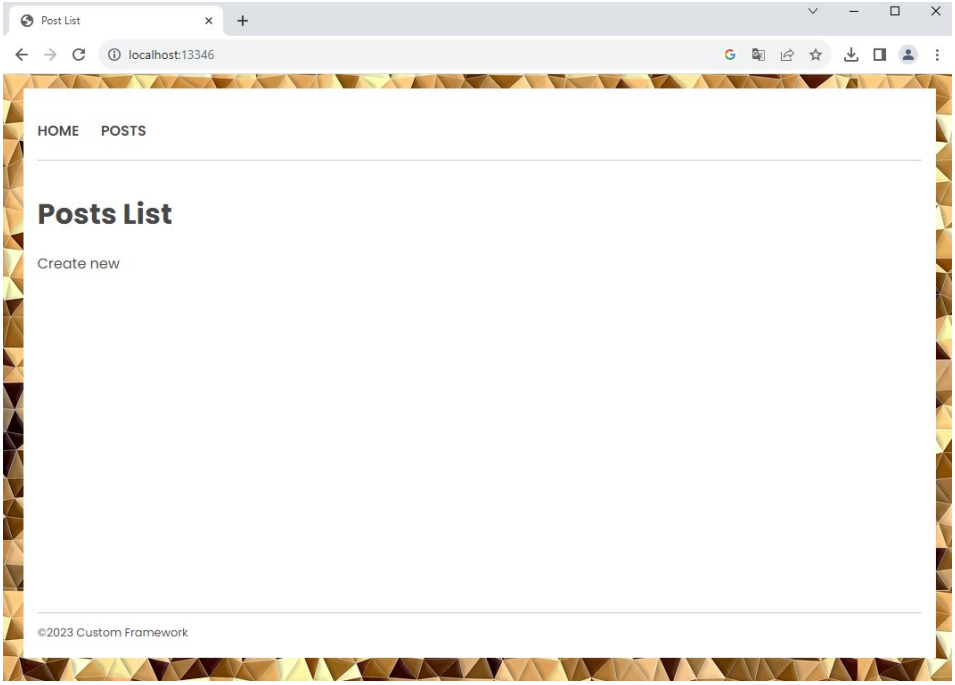


Skopiuj plik `config\config.dist.php` na `config\config.php` i dostosuj ustawienia bazy danych.

Terminalem wejdź do katalogu głównego frameworka i uruchom wbudowany serwer PHP z webrootem w katalogu `public` (jeśli poniżej widać 00000, zaktualizuj swój numer indeksu we właściwościach dokumentu):

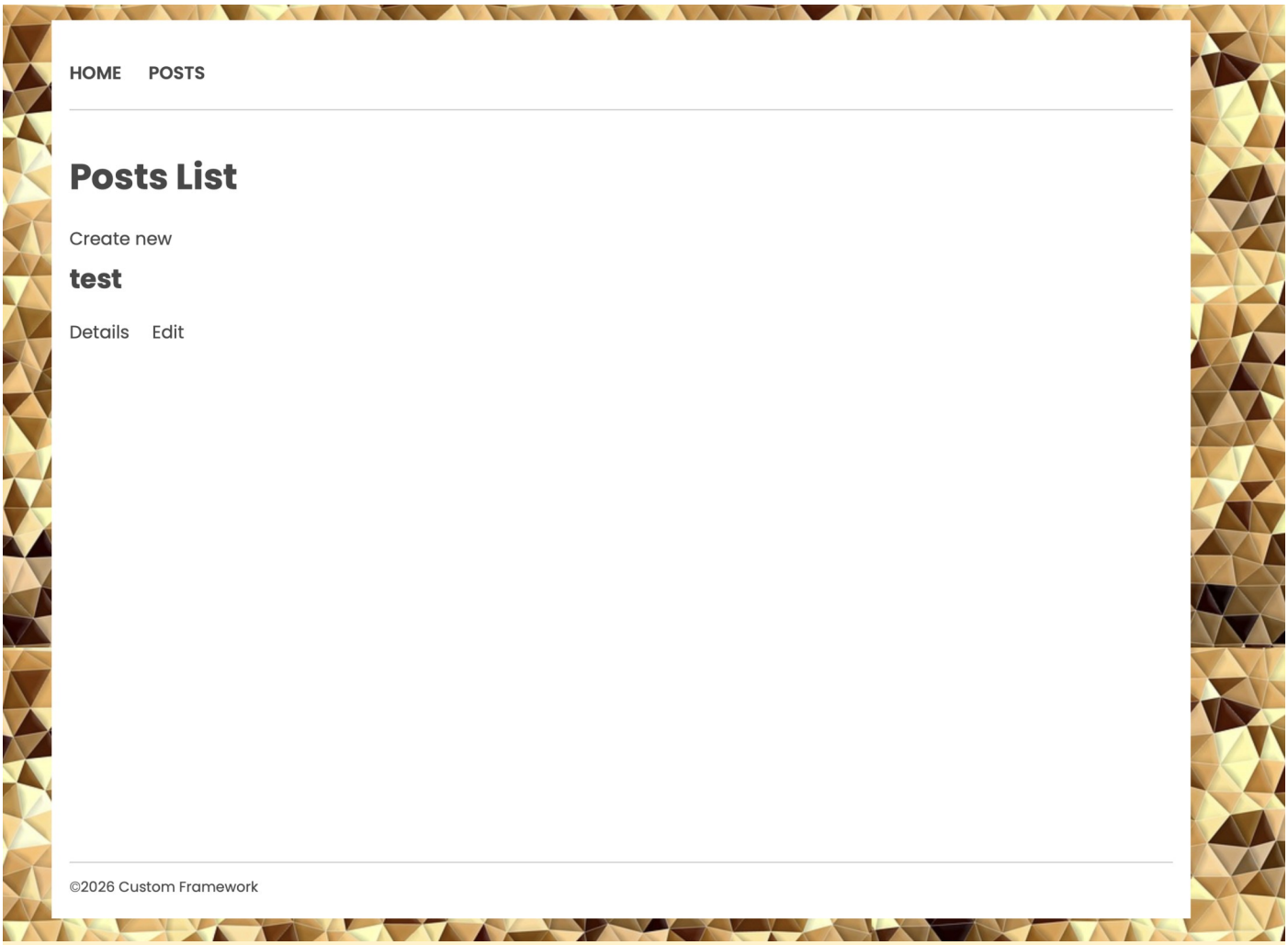
```
php -S localhost:55684 -t .\public\
```

Wejdź przeglądarką na powyższy adres. Framework powinien się wyświetlić:



Utwórz nowy wpis poprzez kliknięcie na `Create new`, wypełnienie przykładowym wpisem z Twoim danymi i wysyłkę formularza.

Wstaw zrzut ekranu listy postów po dodaniu nowego wpisu z Twoimi danymi.



Punkty:	0	1

XDEBUG

Wejdź na stronę https://xdebug.org/wizard. Postępuj zgodnie z instrukcjami, tj:

- skopiuj zawartość strony *http://localhost:<port>/?action=info* do okienka kreatora
- pobierz wskazany plik DLL do odpowiedniego katalogu
- zmodyfikuj zawartość php.ini ze wskazanej ścieżki
- zrestartuj serwer WWW.

Przykładowe ustawienia do wpisania do php.ini:

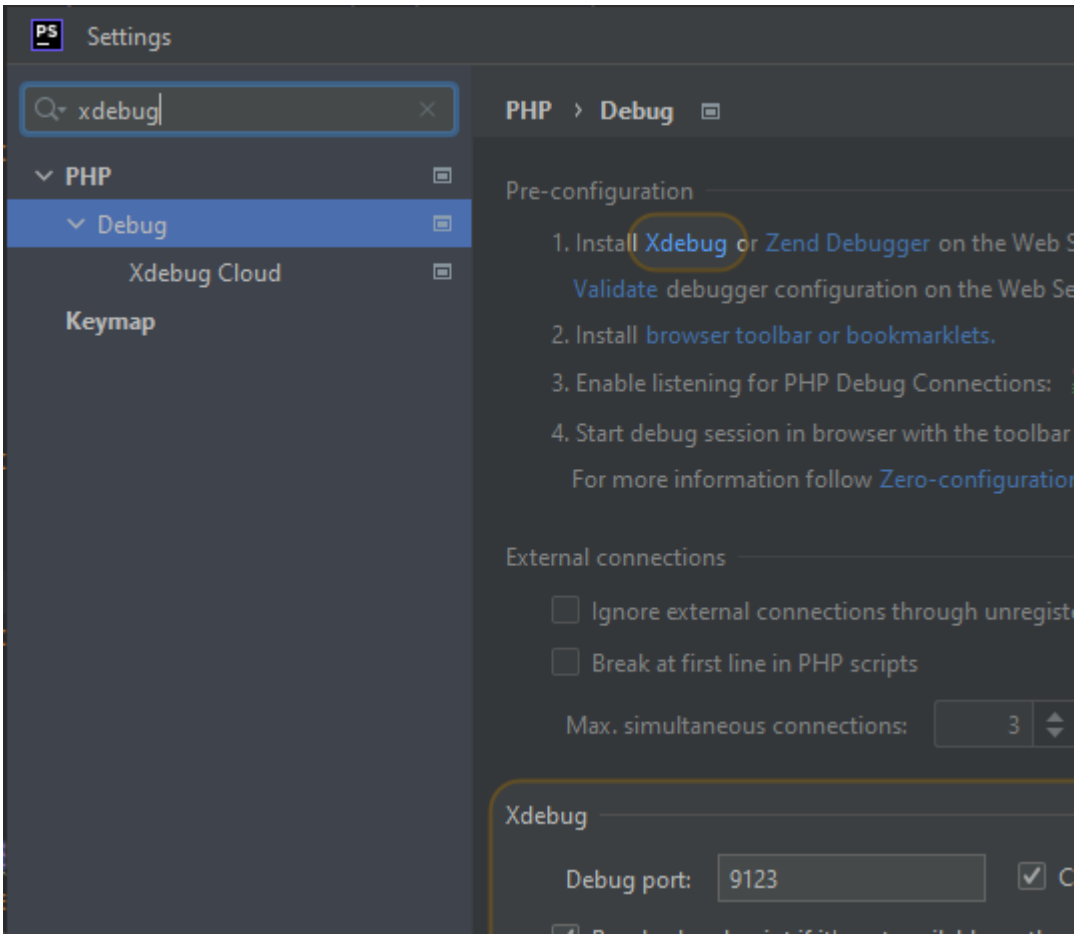
```
zend_extension=xdebug
xdebug.mode=debug
xdebug.discover_client_host=1
xdebug.client_port=9123
```

Uwaga! Wyróżniony powyżej port 9123 należy zmienić na inny, losowy, unikalny. Port musi być unikalny, ponieważ wszyscy pracujemy na jednym serwerze, na takim samym kodzie. W poprzednich edycjach kursu zdarzały się przypadki, że studenci debuggowali nie swoją aplikację 😊

Na stronie PHP INFO powinien pojawić się wpis o XDEBUG:

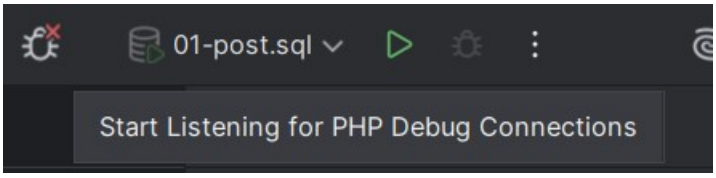


W PhpStorm kliknij File -> Settings (Ctrl + Alt + S). Wyszukaj xdebug. Ustaw port zgodnie z powyżej wylosowanym:



Zatwierdź zmiany.

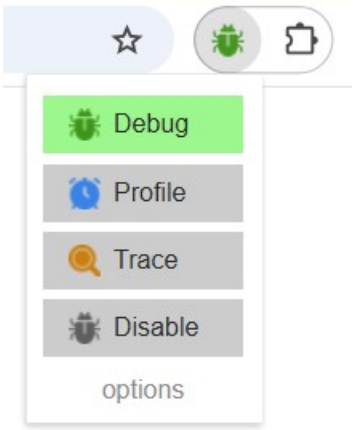
Otwórz plik public/index.php projektu i kliknij obok jednej z pierwszych instrukcji kodu, żeby utworzyć pułapkę (breakpoint). W górnym pasku PhpStorm kliknij ikonkę chrabąszcza (Start Listening (...)):



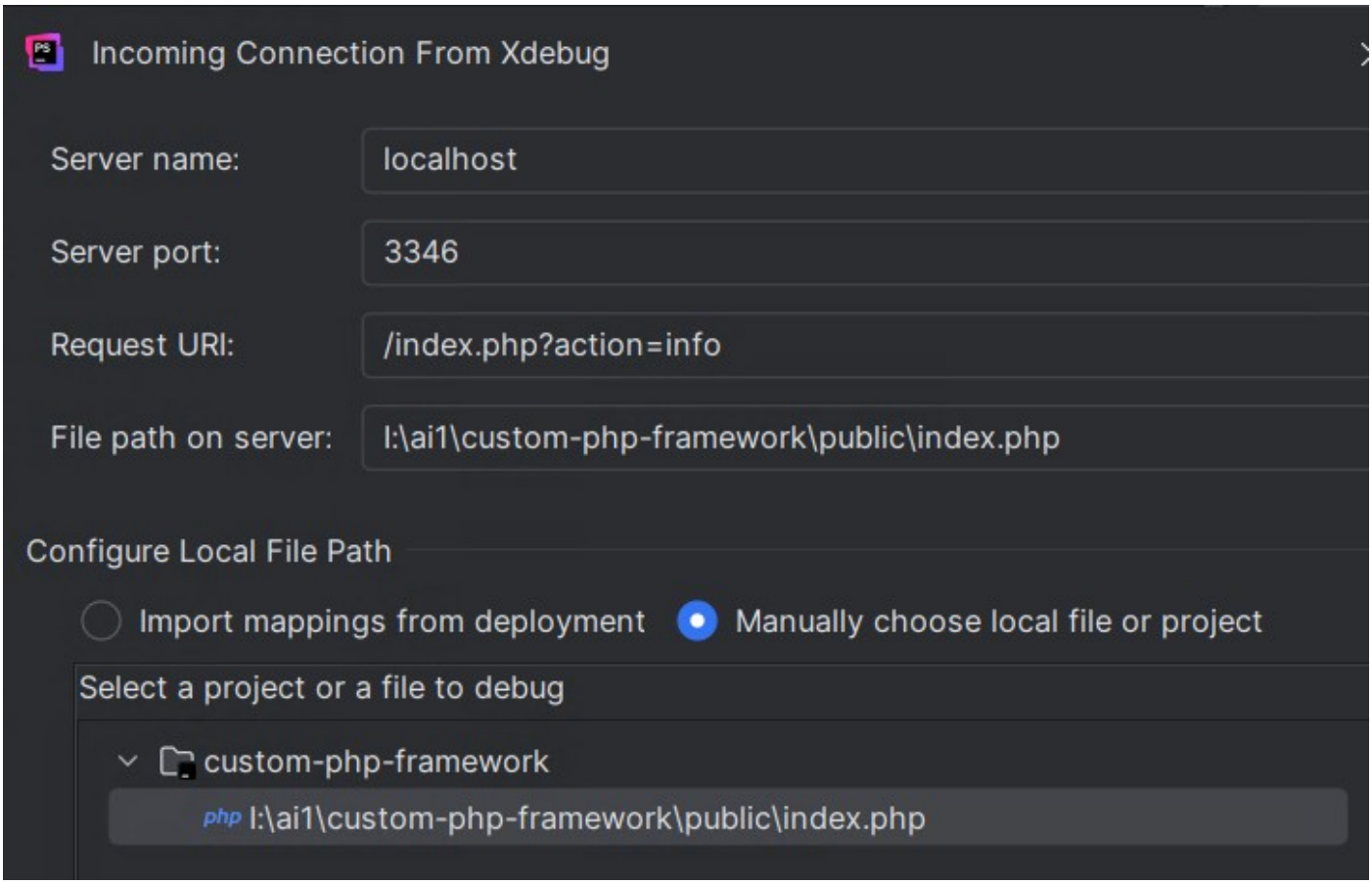
Do Chrome dodaj rozszerzenie Xdebug Helper by JetBrains:



Po zainstalowaniu, wejdź ponownie na stronę frameworka. Kliknij w ikonkę chrabąszcza i zaznacz **Debug**:



Odśwież stronę w przeglądarce. Wykonywanie powinno się zatrzymać, a PhpStorm powinien wyświetlić jednorazowo okno przychodzącego połączenia:



Po zatwierdzeniu, PhpStorm zacznie wyświetlać informacje o stanie zmiennych w pamięci i umożliwi przejście przez aplikację krok po kroku (tzw. step-debugging).

ZATRZYMANIE W PUŁAPCE

Wstaw zrzut ekranu potwierdzający skuteczne zatrzymanie aplikacji na pułapce (breakpoint) w IDE.

```php
<?php
require_once __DIR__ . DIRECTORY_SEPARATOR . '..' . DIRECTORY_SEPARATOR . 'autoload.php';

$config = new \App\Service\Config();

$templating = new \App\Service\Templating();
$router = new \App\Service\Router();

$action = $_REQUEST['action'] ?? null;
switch ($action) {
    case 'post-index':
    case null:
        $controller = new \App\Controller\PostController();
        $view = $controller->indexAction($templating, $router);
        break;
    case 'post-create':
        $controller = new \App\Controller\PostController();
```

- Home
- Posts

Posts List

Create new

- **test**
  - Details
  - Edit

©2026 Custom Framework

Punkty:	0	1

OMÓWIENIE FRAMEWORKA

Połącz się z prowadzącym na spotkaniu TEAMS. Za pomocą udostępniania ekranu, prowadzący omówi strukturę frameworka, obejmującą:

- plik konfiguracyjny config.php i config.dist.php
- organizację assetów – public/assets/src i public/assets/dist
- propozycja organizacji style.less i załączonych styli oraz katalogu public/assets/src/vendor
- znaczenie i propozycja wykorzystania katalogu sql na migracje
- plik index.php – front-controller
- katalog Controller na przykładzie PostController – akcje związane z postami
- przykładowy model w Post.php

- serwisy w katalogu src/Service – czytnik konfiguracji, router, szablony
- szablony i sposób ich dziedziczenia w katalogu templates

Uważnie analizuj pokazywany kod i zadawaj pytania, aby zrozumieć.

Potrzebne dodatkowe wyjaśnienia? W kodzie frameworka są pliki Readme ze szczegółowym opisem konstrukcji 😊

Miejsce na Twoje notatki:

ROZSZERZENIE APLIKACJI

Wymyśl inny model podobny do prezentowanego `Post`. Rozszerz aplikację o implementację swojego modelu oraz związane z nim akcje – lista, prezentacja, tworzenie, edycja, kasowanie.

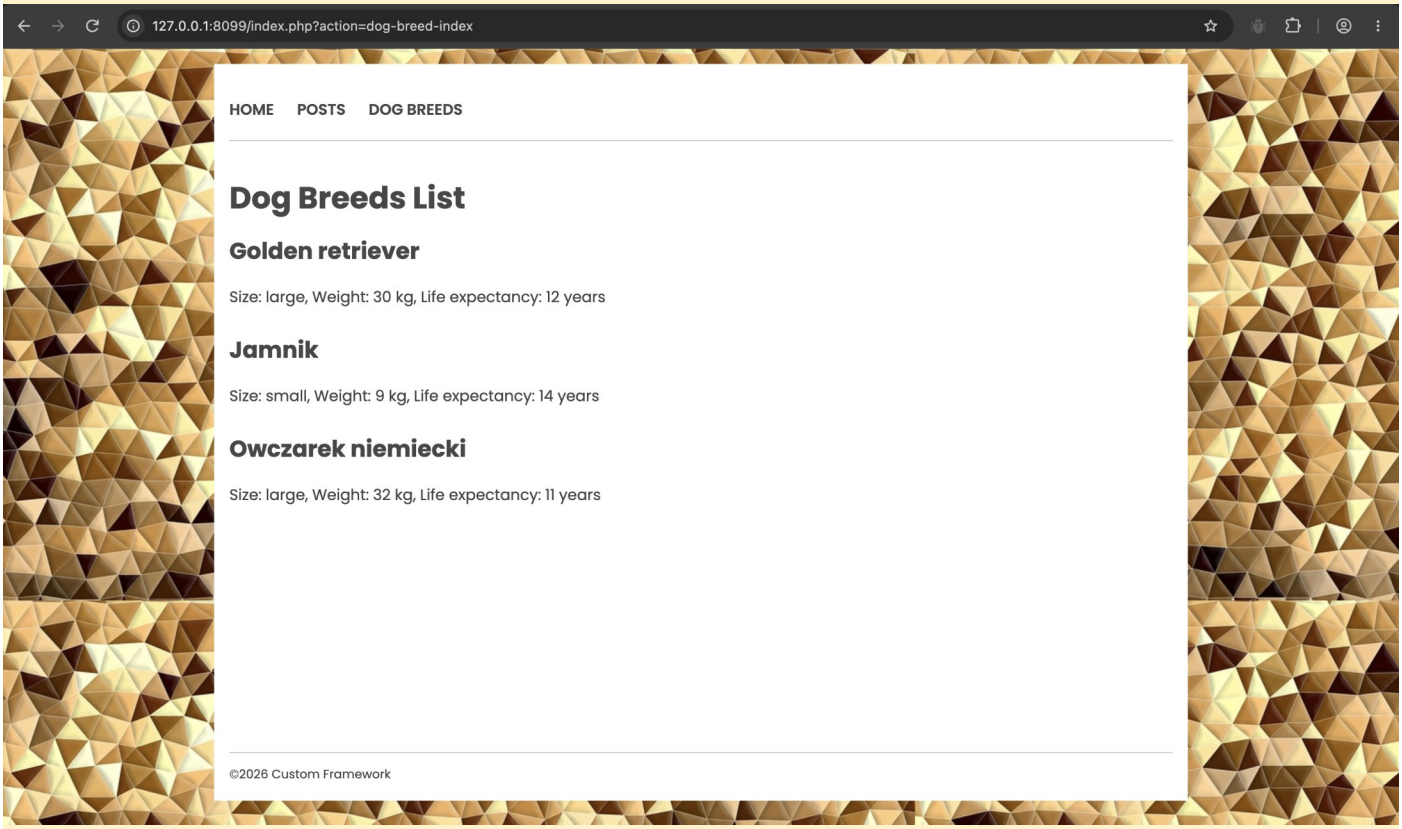
Rozszerzenie a nie zamiana! W poprzednich edycjach kursu wiele osób traciło tu dużo punktów. Twoim zadaniem jest rozszerzyć istniejącą aplikację, czyli dodać nowe funkcjonalności, a nie zastąpić istniejące. To znaczy, że jeśli przykładowo w ramach tego zadania będziesz implementować model `Comment`, to musisz stworzyć osobną klasę `Comment` oraz osobny kontroler `CommentController` i zestaw widoków dla tego kontrolera. Po zakończeniu prac programistycznych, w aplikacji dostępne będą wszystkie strony dotyczące postów, oraz nowe strony dotyczące wybranego modelu (tu komentarzy).

Wstaw zrzut ekranu kodu listy (indexAction):

```php
class DogBreedController
{
    1 usage
    public function indexAction(Templating $templating, Router $router): ?string
    {
        $dogBreeds = DogBreed::findAll();

        return $templating->render( template: 'dog-breed/index.html.php', [
            'dogBreeds' => $dogBreeds,
            'router' => $router,
        ]);
    }
}
```

Wstaw zrzut ekranu listy (z danymi):



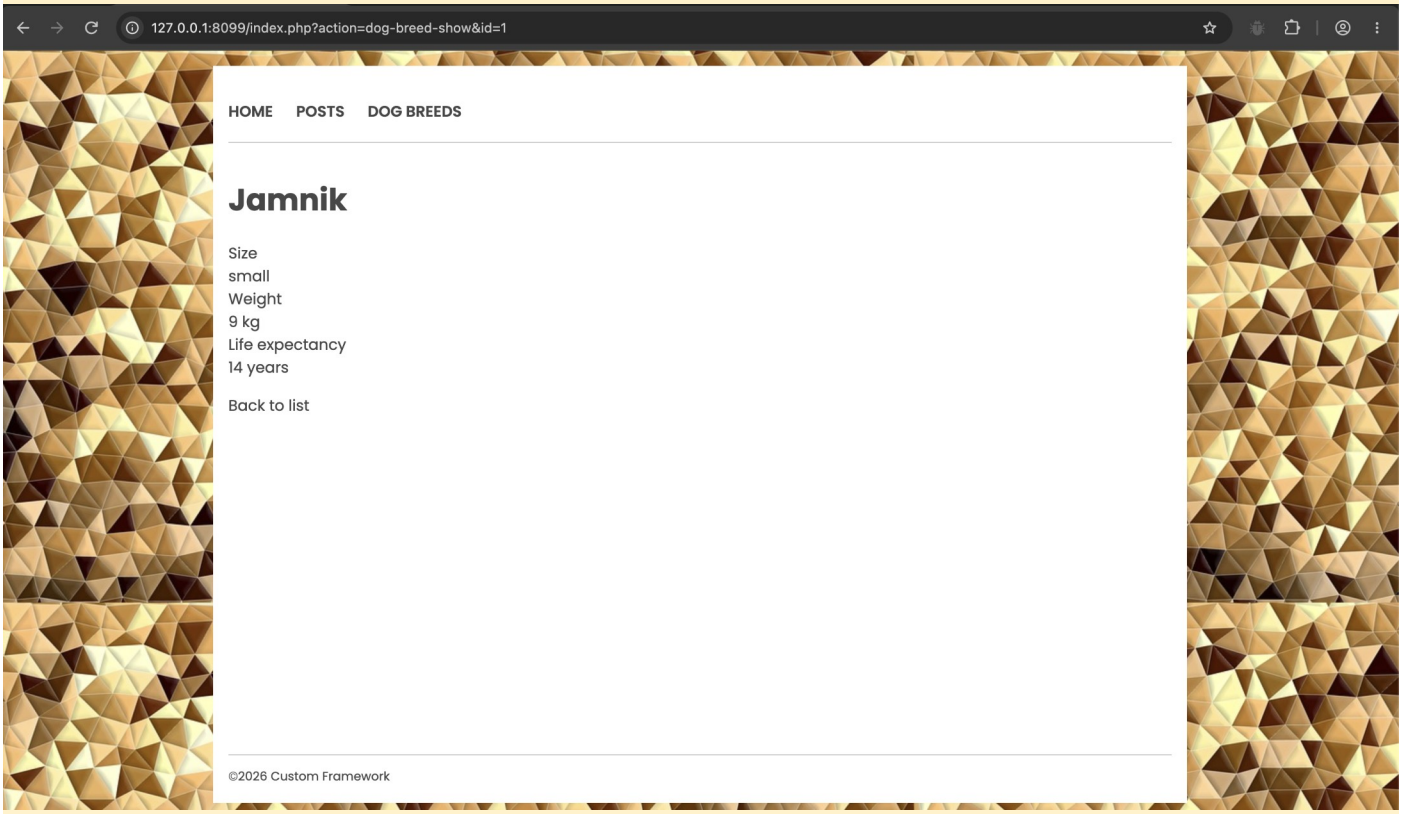
Punkty:	0	1

Wstaw zrzut ekranu kodu podglądu (showAction):

```php
public function showAction(int $dogBreedId, Templating $templating, Router $router): ?string
{
    $dogBreed = DogBreed::find($dogBreedId);
    if (! $dogBreed) {
        throw new NotFoundException( message: "Missing dog breed with id $dogBreedId");
    }

    return $templating->render( template: 'dog-breed/show.html.php', [
        'dogBreed' => $dogBreed,
        'router' => $router,
    ]);
}
```

Wstaw zrzut ekranu podglądu:



Punkty:	0	1

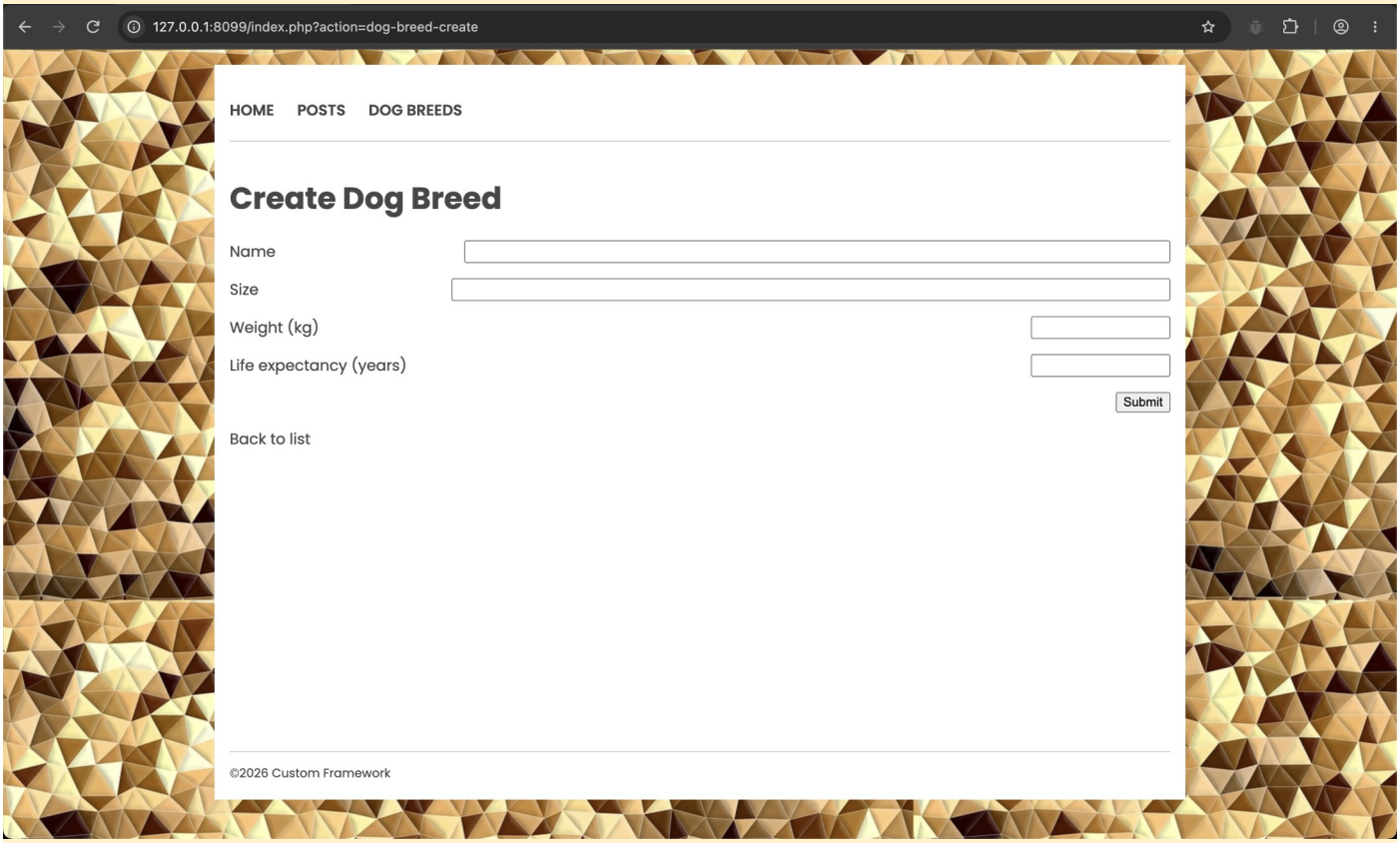
Wstaw zrzut ekranu kodu tworzenia (createAction):

```php
1 usage
public function createAction(?array $requestDogBreed, Templating $templating, Router $router): ?string
{
    if ($requestDogBreed) {
        $dogBreed = DogBreed::fromArray($requestDogBreed);
        $dogBreed->save();

        $router->redirect($router->generatePath( action: 'dog-breed-index'));
        return null;
    }

    return $templating->render( template: 'dog-breed/create.html.php', [
        'dogBreed' => new DogBreed(),
        'router' => $router,
    ]);
}
```

Wstaw zrzut ekranu tworzenia:



Punkty:	0	1

Wstaw zrzut ekranu kodu edycji (editAction):

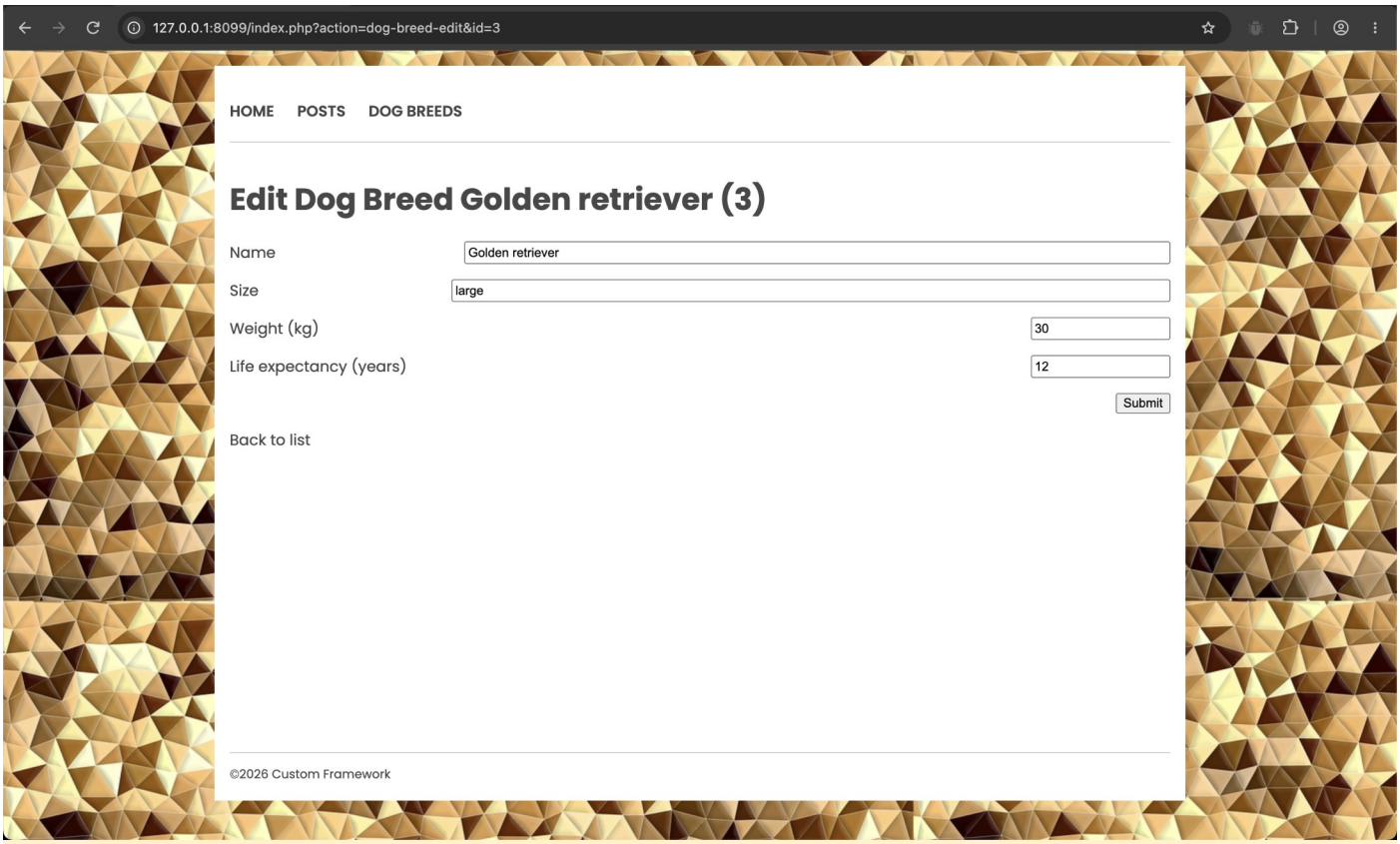
```php
public function editAction(
    int $dogBreedId,
    ?array $requestDogBreed,
    Templating $templating,
    Router $router
): ?string {
    $dogBreed = DogBreed::find($dogBreedId);
    if (! $dogBreed) {
        throw new NotFoundException( message: "Missing dog breed with id $dogBreedId");
    }

    if ($requestDogBreed) {
        $dogBreed->fill($requestDogBreed);
        $dogBreed->save();

        $router->redirect($router->generatePath( action: 'dog-breed-index'));
        return null;
    }

    return $templating->render( template: 'dog-breed/edit.html.php', [
        'dogBreed' => $dogBreed,
        'router' => $router,
    ]);
}
```

Wstaw zrzut ekranu edycji:



Punkty:	0	1

Wstaw zrzut ekranu kodu kasowania (deleteAction):

```php
public function deleteAction(int $dogBreedId, Router $router): ?string
{
    $dogBreed = DogBreed::find($dogBreedId);
    if (! $dogBreed) {
        throw new NotFoundException( message: "Missing dog breed with id $dogBreedId");
    }

    $dogBreed->delete();
    $router->redirect($router->generatePath( action: 'dog-breed-index'));

    return null;
}
```

Punkty:	0	1

COMMIT PROJEKTU DO GIT

Zacommituj i pushnij swoje rozwiązanie do **swojego** repozytorium GIT.

Upewnij się, czy wszystko dobrze się wysłało.

Podaj link do swojego repozytorium:

https://github.com/MichalGudowicz29/MichalGudowicz29.github.io/tree/main

PODSUMOWANIE

W kilku słowach/zdaniach napisz swoje przemyślenia odnośnie tego laboratorium. Nie używaj LLM.

Fajne zadanie trochę kopiowania trochę wklejania ale pozwala zrozumieć strukturę projektów.

Zweryfikuj kompletność sprawozdania. Utwórz PDF i wyślij w terminie.